

CHƯƠNG 4: THIẾT KẾ MẪU

Nhóm mẫu tạo dựng

1. Trong một ứng dụng quản lý cấu hình, bạn cần đảm bảo chỉ có một đối tượng cấu hình duy nhất. Mẫu thiết kế nào phù hợp?
 - A. Factory Method
 - B. Abstract Factory
 - C. Singleton
 - D. Builder
2. Bạn cần tạo ra các loại báo cáo khác nhau (PDF, Excel, Word) từ một giao diện chung. Mẫu thiết kế nào phù hợp?
 - A. Prototype
 - B. Factory Method
 - C. Singleton
 - D. Builder
3. Một ứng dụng cần tạo ra các giao diện người dùng cho các nền tảng khác nhau (Windows, macOS, Linux). Mẫu thiết kế nào phù hợp?
 - A. Prototype
 - B. Factory Method
 - C. Abstract Factory
 - D. Builder
4. Bạn cần xây dựng một đối tượng phức tạp (ví dụ: một chiếc xe) với các tùy chọn khác nhau. Mẫu thiết kế nào phù hợp?
 - A. Prototype
 - B. Factory Method
 - C. Singleton
 - D. Builder
5. Một ứng dụng cần tạo ra các bản sao của một tài liệu mà không cần phải tạo lại từ đầu. Mẫu thiết kế nào phù hợp?
 - A. Prototype
 - B. Factory Method
 - C. Singleton
 - D. Builder
6. Khi nào nên sử dụng mẫu Factory Method thay vì tạo đối tượng trực tiếp bằng new?
 - A. Khi cần tạo ra một đối tượng duy nhất và cần quản lý các đối tượng theo dạng danh sách.
 - B. Khi cần tạo ra các đối tượng có cùng giao diện nhưng khác nhau về cách triển khai.
 - C. Khi cần quản lý các đối tượng theo dạng danh sách.
 - D. Khi cần tạo ra các đối tượng có cấu trúc phức tạp.
7. Tại sao mẫu Builder hữu ích trong việc xây dựng một đối tượng phức tạp?

- A. Nó cho phép tạo ra các đối tượng có cấu trúc phức tạp một cách linh hoạt.
- B. Nó cho phép tạo ra một đối tượng duy nhất.
- C. Nó cho phép quản lý sự phụ thuộc một-nhiều giữa các đối tượng.
- D. Nó cho phép tạo ra các đối tượng theo yêu cầu của người dùng.

8. Bài toán: Bạn đang xây dựng một hệ thống quản lý cấu hình ứng dụng và quyết định áp dụng mẫu thiết kế Singleton.

Câu hỏi: Kết quả chính của việc áp dụng mẫu thiết kế này là gì?

- A. Có rất nhiều các đối tượng cấu hình tồn tại trong suốt vòng đời ứng dụng, đảm bảo tính đa dạng, linh hoạt.
- B. Chỉ có duy nhất một đối tượng cấu hình tồn tại trong suốt vòng đời ứng dụng, đảm bảo tính nhất quán.
- C. Các thành phần khác nhau của ứng dụng có thể có các đối tượng cấu hình riêng biệt.
- D. Việc thay đổi cấu hình sẽ tạo ra một đối tượng cấu hình mới và sẽ có rất nhiều cấu hình tồn tại trong suốt vòng đời của ứng dụng, đảm bảo tính nhất quán của ứng dụng.

9. Bài toán: Bạn đang phát triển một ứng dụng hỗ trợ đa nền tảng và sử dụng mẫu thiết kế Abstract Factory để tạo các thành phần giao diện người dùng.

Câu hỏi: Kết quả chính của việc sử dụng mẫu thiết kế này là gì?

- A. Các thành phần giao diện sẽ được tạo ra một cách độc lập mà không có sự liên kết nào.
- B. Bạn có thể tạo ra các họ thành phần giao diện (ví dụ: bộ nút và thanh cuộn) phù hợp với từng nền tảng một cách nhất quán.
- C. Việc tạo thành phần giao diện sẽ được thực hiện theo từng bước cấu hình.
- D. Bạn không thể tạo ra các họ thành phần giao diện (ví dụ: bộ nút và thanh cuộn) phù hợp với từng nền tảng một cách nhất quán..

10. Bài toán: Áp dụng Prototype để tạo các đơn vị quân trong một trò chơi chiến thuật (ví dụ: tạo nhiều lính bộ binh giống nhau từ một nguyên mẫu) sẽ mang lại kết quả gì?

- A. Mỗi đơn vị quân sẽ được tạo ra với các thuộc tính hoàn toàn ngẫu nhiên, các đơn vị quân sẽ chia sẻ cùng một đối tượng nguyên mẫu.
- B. Việc tạo các đơn vị quân mới sẽ nhanh hơn và tiết kiệm tài nguyên hơn so với việc khởi tạo từ đầu.
- C. Việc tạo các đơn vị quân mới sẽ đẹp hơn, chuyên nghiệp hơn và tiết kiệm tài nguyên hơn so với việc khởi tạo từ đầu.
- D. Việc tạo đơn vị quân sẽ yêu cầu một quy trình xây dựng phức tạp để thiết lập các thuộc tính.

Nhóm mẫu cấu trúc

1. Trong một ứng dụng cần tích hợp một thư viện bên ngoài có giao diện khác với giao diện của ứng dụng, mẫu thiết kế nào phù hợp?
 - A. Bridge
 - B. Composite
 - C. Adapter
 - D. Decorator
2. Một ứng dụng cần chạy trên nhiều nền tảng khác nhau (Windows, macOS, Linux) mà không cần thay đổi nhiều code, mẫu thiết kế nào phù hợp?
 - A. Bridge
 - B. Composite
 - C. Adapter
 - D. Decorator
3. Một ứng dụng cần trì hoãn việc tải một hình ảnh lớn cho đến khi người dùng thực sự muốn xem nó, mẫu thiết kế nào phù hợp?
 - A. Facade
 - B. Flyweight
 - C. Proxy
 - D. Adapter
4. Tại sao mẫu Adapter hữu ích khi tích hợp thư viện bên ngoài?
 - A. Nó giúp tạo ra các đối tượng có cùng giao diện.
 - B. Nó giúp chuyển đổi giao diện của thư viện bên ngoài để phù hợp với ứng dụng.
 - C. Nó giúp tạo ra một đối tượng duy nhất.
 - D. Nó giúp quản lý các đối tượng theo dạng cây phân cấp.
5. Một lớp cần che giấu sự phức tạp của nhiều lớp bên trong và cung cấp một giao diện dễ sử dụng hơn cho người dùng, mẫu thiết kế nào phù hợp?
 - A. Facade
 - B. Flyweight
 - C. Proxy
 - D. Adapter
6. Bài toán: Bạn có một hệ thống xử lý dữ liệu từ một nguồn bên ngoài với định dạng khác biệt. Bạn áp dụng mẫu thiết kế Adapter.

Câu hỏi: Kết quả chính của việc áp dụng mẫu thiết kế này là gì?

 - A. Tạo ra một giao diện mới hoàn toàn cho hệ thống của bạn.
 - B. Cho phép hệ thống của bạn làm việc với dữ liệu từ nguồn bên ngoài mà không cần thay đổi cấu trúc hiện tại.
 - C. Cho phép hệ thống của bạn làm việc với dữ liệu từ nguồn bên trong mà không cần thay đổi cấu trúc hiện tại.

- D.** Cho phép hệ thống của bạn làm việc với dữ liệu từ nguồn bên ngoài có thể cần thay đổi cấu trúc hiện tại của hệ thống.
7. Bài toán: Bạn đang xây dựng một ứng dụng có thể chạy trên nhiều hệ điều hành khác nhau và sử dụng mẫu thiết kế Bridge.
- Câu hỏi: Kết quả quan trọng của việc áp dụng mẫu thiết kế này là gì?
- A.** Ứng dụng sẽ có giao diện người dùng giống hệt nhau trên tất cả các nền tảng.
 - B.** Phần logic nghiệp vụ của ứng dụng có thể thay đổi độc lập với phần triển khai cho từng hệ điều hành.
 - C.** Phần logic nghiệp vụ của ứng dụng có thể thay đổi nhưng liên quan mật thiết với phần triển khai cho từng hệ điều hành.
 - D.** Ứng dụng sẽ có hiệu suất tối ưu trên một nền tảng cụ thể.
8. Bài toán: Bạn muốn thêm các tính năng bổ sung (ví dụ: mã hóa, nén) vào một đối tượng luồng dữ liệu mà không muốn tạo ra các lớp con mới. Bạn áp dụng mẫu thiết kế Decorator.
- Câu hỏi: Kết quả quan trọng của việc sử dụng Decorator trong trường hợp này là gì?
- A.** Bạn có thể thêm và nhưng không thể loại bỏ các trách nhiệm một cách động cho các đối tượng.
 - B.** Bạn có thể thêm và loại bỏ các trách nhiệm một cách động cho các đối tượng.
 - C.** Hiệu suất của luồng dữ liệu được cải thiện đáng kể.
 - D.** Bạn có giao diện của luồng dữ liệu trở nên đơn giản hơn.

Nhóm mẫu hành vi

1. Trong một ứng dụng GUI, bạn cần xử lý các sự kiện khác nhau (nhấp chuột, gõ phím) bằng các đối tượng xử lý khác nhau. Mẫu thiết kế nào phù hợp?
 - A. Command
 - B. Interpreter
 - C. Chain of Responsibility
 - D. Iterator
2. Bạn cần xây dựng một bộ thông dịch cho một ngôn ngữ truy vấn cơ sở dữ liệu đơn giản. Mẫu thiết kế nào phù hợp?
 - A. Command
 - B. Interpreter
 - C. Chain of Responsibility
 - D. Iterator
3. Một đối tượng cần thay đổi hành vi của nó dựa trên trạng thái bên trong của nó. Mẫu thiết kế nào phù hợp?
 - A. Mediator
 - B. Memento
 - C. Observer
 - D. State
4. Bạn cần xử lý một đơn đặt hàng, nơi các bước như "xác nhận đơn hàng", "thanh toán", "giao hàng" có thể được ghi đè bởi các lớp con để xử lý các loại đơn hàng khác nhau. Mẫu thiết kế nào phù hợp?
 - A. Strategy
 - B. Template Method
 - C. Visitor
 - D. Command
5. Bạn cần thực hiện chức năng "undo" trong một ứng dụng chỉnh sửa ảnh. Mẫu thiết kế nào phù hợp?
 - A. Mediator
 - B. Memento
 - C. Observer
 - D. State
6. Bài toán: Bạn đang xây dựng một hệ thống xử lý yêu cầu hỗ trợ nhiều loại yêu cầu khác nhau. Bạn áp dụng mẫu thiết kế Chain of Responsibility.

Câu hỏi: Kết quả chính của việc áp dụng mẫu thiết kế này là gì?

 - A. Mỗi loại yêu cầu sẽ có bộ xử lý riêng và yêu cầu được chuyển qua một chuỗi đối tượng phù hợp.
 - B. Yêu cầu được chuyển qua một chuỗi đối tượng, đối tượng phù hợp sẽ xử lý.

- C.** Tất cả yêu cầu do một đối tượng xử lý, mỗi yêu cầu sẽ được xử lý phù hợp riêng.
 - D.** Yêu cầu bị bỏ qua nếu không có đối tượng xử lý.
- 7. Bài toán: Bạn đang xây dựng một hệ thống quản lý trạng thái cho các đối tượng game (ví dụ: nhân vật có trạng thái "đang di chuyển", "đang tấn công"). Bạn áp dụng mẫu thiết kế State.
Câu hỏi: Kết quả chính của việc áp dụng mẫu thiết kế này là gì?
 - A.** Hành vi cố định và hành vi thay đổi theo trạng thái.
 - B.** Hành vi thay đổi theo trạng thái.
 - C.** Trạng thái thông báo cho mọi đối tượng.
 - D.** Trạng thái lưu trữ bên ngoài.
- 8. Bài toán: Trong một hệ thống giao tiếp giữa nhiều thành phần độc lập, bạn sử dụng mẫu thiết kế Observer.
Câu hỏi: Kết quả mong muốn của việc sử dụng Observer là gì?
 - A.** Thay đổi trạng thái các thành phần giao tiếp trực tiếp.
 - B.** Thay đổi trạng thái một thành phần, các thành phần liên quan được thông báo.
 - C.** Thay đổi trạng thái các thành phần trung gian, các thành phần liên quan được thông báo.
 - D.** Thành phần chỉ nhận thông báo khi yêu cầu.